GUION DE EXPOSICIÓN ORAL Y DEFENSA ACADÉMICA

FitManage Pro – Sistema Integral de Gestión para Club Deportivo (Java 21 SE)

Cátedra: *Programación Orientada a Objetos / Laboratorio de Software*
Integrantes y Expositores: *Juan, Rocio, Merlina, Alfonsina, Sara, Joel*
Duración Máxima de Exposición: *20 minutos (18:30 discurso neto + 01:30 buffer de preguntas)*
Versión Oficial: *1.0.0 LTS - Convocatoria Noviembre 2026*
Entregables Vinculados: `presentacion_fitmanage.html`, `UML_DIAGRAM.md`, `PLAN_PRESENTACION_SLIDES.md`

CRONOGRAMA MAESTRO Y TIEMPOS POR EXPOSITOR

BLOQUE	EXPOSITOR	MÓDULO PRINCIPAL Y EJE TEMÁTICO	DIAPOSITIVAS	TIEMPO ASIGNADO
01	JUAN	Apertura, Visión, Arquitectura 4 Capas, Jerarquía `Persona`	Slides 01, 02, 03, 04	00:00 - 03:15 (3m 15s)
02	ROCIO	Dominio Deportivo, Composición Fuerte (`Rutina *-- Ejercicio`), Tonelaje	Slide 05	03:15 - 06:15 (3m 00s)
03	MERLINA	Clases Grupales, Agregación Débil (`o--`), Aforo y `CupoAgotadoException`	Slide 06	06:15 - 09:15 (3m 00s)
04	ALFONSINA	Facturación Polimórfica (`Facturable`), Membresías, Antigüedad (10%)	Slide 07	09:15 - 12:15 (3m 00s)
05	SARA	GUI Java Swing, CardLayout, ThemeConfig, Badges y Molinete Virtual	Slides 08, 09	12:15 - 15:15 (3m 00s)
06	JOEL	Persistencia Atómica JSON, Tests (7/7), Graphify y Cierre Grupal	Slides 10, 11, 12, 13	15:15 - 18:30 (3m 15s)
07	EQUIPO	Buffer de Contingencia y Ronda Abierta de Preguntas del Jurado	Slide 13	18:30 - 20:00 (1m 30s)

BLOQUE 1: JUAN — APERTURA, ARQUITECTURA Y JERARQUÍA PERSONA

Tiempo asignado: 03 minutos 15 segundos (00:00 a 03:15)
Diapositivas: `Slide 01: Portada Ejecutiva` | `Slide 02: Dominio y 4 Capas` | `Slide 03: Diagrama UML Integral` | `Slide 04: Jerarquía Persona`
Clases Java Clave: `com.clubdeportivo.model.Persona`, `com.clubdeportivo.model.Socio`, `com.clubdeportivo.model.Autenticable`, `com.clubdeportivo.model.TipoAcceso`

Acotaciones Escénicas Iniciales

- [00:00] *Juan se ubica en el centro del estrado, postura erguida, contacto visual sereno con los miembros del jurado. Tono seguro, cordial y profesional.*
- [00:08] *Avanza a la Slide 02 con click o tecla Flecha Derecha. Señala el flujo unidireccional de capas.*
- [01:10] *Avanza a la Slide 03. Permite 3 segundos de apreciación del gran Diagrama UML vectorial y sitúa el cuadrante superior izquierdo.*
- [01:50] *Avanza a la Slide 04. Modula la voz hacia un tono analítico al justificar la abstracción ontológica.*

Discurso de Exposición

[00:00 - 00:45] Introducción y Planteo del Problema
"Buenos días profesores y miembros del tribunal evaluador. Junto a mi equipo —Rocio, Merlina, Alfonsina, Sara y Joel— nos complace presentar **FitManage Pro**, una solución integral de software diseñada para transformar la gestión deportiva y administrativa de un club moderno.

En el contexto actual, los centros de entrenamiento sufren cuellos de botella críticos: falta de trazabilidad en las cuotas sociales, sobreventa de cupos en salones de fitness, descontrol en el ingreso físico y pérdida de planes de entrenamiento. Frente a esto, nos planteamos un desafío de ingeniería riguroso: construir un sistema robusto, reactivo, sin dependencias externas pesadas, desarrollado 100% en **Java 21 SE**, que exhiba la máxima pureza en principios de Programación Orientada a Objetos."

[00:45 - 01:30] Arquitectura en 4 Capas y Desacoplamiento
"Para estructurar este sistema, concebimos una arquitectura multicapa estrictamente desacoplada: 1. En la cúspide, la **Capa de Presentación (GUI)**, implementada con Java Swing nativo bajo el patrón CardLayout. 2. La **Capa de Servicios de Negocio (`service`)**, que orquesta la lógica transaccional, valida accesos y computa liquidaciones. 3. La **Capa de Acceso a Datos o Repositorios (`repository`)**, que define contratos mediante la interfaz genérica `IRepository<T>`. 4. Y en la base, nuestro **Dominio Puro (`model`)** respaldado por un motor de persistencia atómica en JSON.

Como observan en nuestro Diagrama de Clases UML completo en pantalla, la separación de responsabilidades es absoluta: la vista nunca interactúa directamente con los archivos físicos, garantizando alta cohesión y bajo acoplamiento."

[01:30 - 02:45] Jerarquía de Herencia: La Clase Base Abstracta `Persona`
"Centrándonos en el modelado ontológico de los actores, diseñamos la clase abstracta `Persona`. En el mundo real de un club no existe una 'persona genérica'; toda entidad que interactúa es un `Socio` activo o un `Instructor` profesional. Por ello, `Persona` se declara `abstract`, impidiendo su instanciación aislada.

Aplicamos un encapsulamiento metódico: sus atributos troncales — `id`, `dni`, `nombre`, `apellido`, `email`, `telefono` y fechas clave— están definidos con modificador de acceso `protected (#)`. Esto permite que las

subclases `Socio` e `Instructor` hereden y operen directamente sobre su estado identitario sin exponerlo indiscriminadamente al resto del sistema.

Asimismo, `Persona` provee comportamiento concreto compartido como `calcularEdad()`, resolviendo la diferencia cronológica con `LocalDate.now()` sin duplicación de código. Y para forzar el polimorfismo dinámico en tiempo de compilación, definimos métodos abstractos como `mostrarDetalle()`, `getRol()` y `validarIngreso(TipoAcceso tipo)`. Cada subclase asume la responsabilidad ineludible de implementar su propia lógica de validación e impresión."

[02:45 - 03:15] Síntesis y Transición
"De este modo, `Socio` implementa las reglas de cuota al día, mientras que `Instructor` exhibe sus legajos y especialidad deportiva. Pero un socio no solo asiste al club: entrena mediante cargas físicas dosificadas. Para profundizar en cómo modelamos el entrenamiento mediante composición fuerte, le cedo la palabra a mi compañera Rocio."

Defensa de Cátedra (Pregunta Típica del Jurado)

> **Profesor:** *"¿Por qué declararon `Persona` como clase `abstract` en lugar de una interfaz o una clase concreta? ¿Por qué no usar composición con el patrón Role o State para modelar los roles de Socio e Instructor?"*
> **Respuesta de Juan:** *"Declaramos `Persona` abstracta porque en el dominio ontológico de un club deportivo no existe una 'persona genérica'; todo individuo que ingresa al sistema tiene un rol concreto e intransferible. Instanciar `new Persona()` carece de semántica de negocio. La herencia nos permite reutilizar estado común fuertemente tipado (`id`, `dni`, `nombre`, `email`, `fechaNacimiento`) y comportamiento concreto compartido como `calcularEdad()`, mientras forzamos polimórficamente a través de métodos abstractos como `mostrarDetalle()` que cada subclase exhiba sus particularidades. Si bien el patrón State o Decorator permitiría mutar de rol en tiempo de ejecución, en este alcance académico la especialización rígida garantiza integridad referencial, tipado seguro y prevención de estados inconsistentes en tiempo de compilación."*

BLOQUE 2: ROCIO — MÓDULO DE RUTINAS, COMPOSICIÓN FUERTE Y TONELAJE

Tiempo asignado: 03 minutos 00 segundos (03:15 a 06:15)
Diapositivas: `Slide 05: Composición Fuerte: Rutina, Ejercicios y Cálculo de Tonelaje`
Clases Java Clave: `com.clubdeportivo.model.Rutina`, `com.clubdeportivo.model.Ejercicio`, `com.clubdeportivo.model.GrupoMuscular`, `com.clubdeportivo.service.RutinaService`

Acotaciones Escénicas

- [03:15] *Rocio toma la posta con seguridad, da un paso al frente y apunta con el puntero al rombo negro sólido en la diapositiva 05.*
- [04:10] *Enfoca la mirada en el docente al enunciar la fórmula matemática de tonelaje acumulado.*
- [05:20] *Muestra naturalidad y fluidez técnica al relacionar la composición UML con el ciclo de vida de los objetos en memoria.*

Discurso de Exposición

[03:15 - 04:00] Composición Fuerte vs Agregación: Definición Formal
"Muchas gracias Juan. Al abordar el subsistema de acondicionamiento físico, nos encontramos con una decisión de diseño fundamental en UML: ¿cómo se relaciona una `Rutina` de entrenamiento con los `Ejercicios` que la componen?

Optamos deliberadamente por una **Composición Fuerte**, representada en el diagrama con un **rombo negro relleno** en el extremo de `Rutina` (`Rutina 1 *-- 1..* Ejercicio`). En términos de POO, la composición denota una relación 'parte-todo' caracterizada por la dependencia existencial irrestricta de las partes respecto del todo. Un `Ejercicio` dentro de nuestro modelo no representa un concepto de enciclopedia estática; representa una prescripción técnica individualizada: series, repeticiones planificadas, carga en kilogramos y segundos de descanso para un socio particular."

[04:00 - 04:50] Ciclo de Vida y Encapsulamiento en Memoria
"¿Qué consecuencias prácticas tiene esta decisión en el código Java?
En primer lugar, el ciclo de vida está indisolublemente ligado: un `Ejercicio` no tiene razón de existir de manera autónoma fuera de su `Rutina`. Si una rutina es eliminada del sistema o dada de baja por un instructor, todas sus instancias anidadas de `Ejercicio` pierden su anclaje en el grafo de objetos y son recolectadas automáticamente por el Garbage Collector de la JVM.

En segundo lugar, la `Rutina` actúa como la Raíz del Agregado (*Aggregate Root*). El acceso a la lista interna de ejercicios se encuentra estrictamente encapsulado: no exponemos colecciones mutables directamente. La clase `Rutina` provee métodos protectores como `agregarEjercicio(Ejercicio e)` y `eliminarEjercicio(String ejercicioId)`, los cuales validan que la rutina mantenga siempre un estado consistente."

[04:50 - 05:45] Algoritmo de Negocio: Cálculo de Tonelaje Global
"Esta cohesión estructural nos permite implementar métricas biométricas de alto valor para los preparadores físicos. La clase `Ejercicio` implementa el método `calcularVolumenTotal()`, el cual computa: $$\text{Volumen Ejercicio} = \text{Series} \times \text{Repeticiones} \times \text{PesoSugeridoKg}$$

A su vez, la `Rutina` delega polimórficamente este cálculo e itera sobre su composición interna mediante el método `calcularVolumenGlobal()`. Si un socio ejecuta 4 series de 10 repeticiones con 100 kg en Press de

Banca, y luego 3 series de 12 repeticiones con 20 kg en Aperturas, la rutina totaliza dinámicamente: $$(4 \times 10 \times 100) + (3 \times 12 \times 20) = 4.000 + 720 = 4.720\text{ kg de volumen acumulado}.$$ Este cálculo es evaluado en tiempo real en nuestra suite automatizada, garantizando precisión numérica absoluta."

[05:45 - 06:15] Cierre y Pase a Agregación Débil
"Como vemos, la composición fuerte garantiza integridad y encapsulamiento ontológico. Sin embargo, en el club existen relaciones donde los objetos deben preservar su total independencia existencial. Para analizar la contracara de esta relación en las Clases Grupales, le doy la palabra a Merlina."

Defensa de Cátedra (Pregunta Típica del Jurado)

> **Profesor:** *"¿Qué diferencia práctica hay entre agregación y composición en este caso? Si elimino una Rutina en el sistema, ¿qué ocurre con sus ejercicios a nivel de persistencia en disco y a nivel de memoria?"*
> **Respuesta de Rocio:** *"La composición modela una relación 'parte-todo' con ciclo de vida coincidente. En memoria, la clase `Rutina` instancia y contiene su lista interna `List<Ejercicio> ejercicios`. Si la instancia de `Rutina` es eliminada, sus ejercicios pierden sus referencias vivas y el Garbage Collector de Java libera su espacio. A nivel de persistencia en disco, en nuestro archivo `rutinas.json`, los ejercicios no residen en una tabla o archivo independiente; están embebidos como nodos hijos JSON dentro del objeto padre Rutina. Al borrar la rutina, su subárbol JSON desaparece atómicamente. Si hubiéramos empleado agregación débil, el ejercicio tendría una identidad global independiente y persistiría en un catálogo general, lo cual distorsionaría el dominio porque una prescripción de 4x10 a 80kg es privativa de ese plan."*

BLOQUE 3: MERLINA — CLASES GRUPALES, AGREGACIÓN DÉBIL Y EXCEPCIONES

Tiempo asignado: 03 minutos 00 segundos (06:15 a 09:15)
Diapositivas: `Slide 06: Agregación Débil, Control de Aforo y Manejo de Excepciones`
Clases Java Clave: `com.clubdeportivo.model.ClaseGrupal`, `com.clubdeportivo.model.CupoAgotadoException`, `com.clubdeportivo.model.Disciplina`, `com.clubdeportivo.service.ClaseService`

Acotaciones Escénicas

- [06:15] *Merlina asume el control del escenario. Señala en la diapositiva 06 los rombos blancos vacíos entre ClaseGrupal, Instructor y Socio.*
- [07:20] *Eleva el tono con énfasis didáctico al explicar la diferencia entre devolver `boolean` o lanzar una excepción de negocio.*
- [08:30] *Presenta con orgullo el fragmento de código de `CupoAgotadoException`.*

Discurso de Exposición

[06:15 - 07:05] Agregación Débil: Independencia de Ciclos de Vida
"Muchas gracias Rocio. En contraste directo con la composición recién explicada, el módulo de actividades grupales requería modelar una relación no destructiva. En un club, una clase colectiva de Spinning, Yoga o CrossFit congrega a un `Instructor` que la coordina y a múltiples `Socios` que asisten a ella.

En UML modelamos esto mediante **Agregación Débil**, representada por el **rombo blanco transparente o hueco** en el extremo de `ClaseGrupal` (`ClaseGrupal 1 o-- 0..1 Instructor` y `ClaseGrupal 1 o-- 0..* Socio`).
¿Por qué agregación y no composición? Porque las partes poseen ciclos de vida completamente independientes del todo. Si el club suspende una clase de Spinning por mantenimiento de sala o lluvia, el profesor no es despedido de la institución ni los socios inscriptos son dados de baja de la base de datos. La clase únicamente referencia identidades que existen previamente por derecho propio."

[07:05 - 08:00] Regla de Negocio: Control de Aforo y Aislamiento
"La clase `ClaseGrupal` custodia celosamente sus invariantes de aforo. Cada actividad cuenta con atributos como `cupoMaximo`, `disciplina`, `diaSemana`, `horaInicio` y una lista interna `List<Socio> sociosInscriptos`.

Para inscribir a un miembro, la clase expone el método:

```
public synchronized boolean inscribirSocio(Socio socio) throws CupoAgotadoException
```

El método opera bajo una verificación preventiva: primero comprueba que el socio no se encuentre inscripto previamente. Luego, evalúa si la cantidad actual de inscriptos ha alcanzado o superado el `cupoMaximo`. Si aún restan cupos, incorpora la referencia y actualiza las vacantes. Pero si el límite físico de la sala está saturado, el sistema no recurre a soluciones frágiles."

[08:00 - 08:45] La Excepción de Negocio: `CupoAgotadoException`
"En lugar de devolver un simple valor `false` o silenciar el problema, diseñamos la excepción verificada de negocio `CupoAgotadoException`, la cual extiende directamente de `java.lang.Exception`.

Esta decisión responde al principio de **Fallo Rápido (Fail-Fast)** y hace explícito el contrato en tiempo de compilación. Cuando el cupo se satura, el método aborta la operación y lanza:

```
throw new CupoAgotadoException("Cupo agotado para la clase: " + nombre + " (Máx: " + cupoMaximo +
```

Esto fuerza a la capa superior —sea nuestro `ClaseService` o el controlador visual `ClasesPanel`— a capturar formalmente el escenario mediante bloques `try-catch`, registrando la advertencia y desplegando un diálogo de alerta rojo en la interfaz sin corromper los datos."

[08:45 - 09:15] Conexión y Pase a Finanzas
"Este control estricto de excepciones fue probado exitosamente en nuestro test unitario número 3, donde intentamos forzar la inscripción de un 4to socio en una clase limitada a 3 plazas, capturando la excepción esperada. Ahora bien, para que un socio disfrute de estas clases, debe mantener regularizada su condición arancelaria. Le cedo la palabra a Alfonsina para explicar el módulo de Facturación y Membresías."

Defensa de Cátedra (Pregunta Típica del Jurado)

> **Profesor:** *"¿Por qué implementaron `CupoAgotadoException` como una excepción verificada (`checked`) en lugar de devolver `false` o usar `RuntimeException` / `IllegalStateException`?"*
> **Respuesta de Merlina:** *"Optamos por una excepción verificada de dominio (`checked exception`) para forzar el tratamiento del error en tiempo de compilación según las buenas prácticas de Java. Si devolviéramos un booleano `false`, un programador en la capa de interfaz podría omitir involuntariamente la comprobación del valor de retorno (`if (!resultado)`), permitiendo que el flujo continúe como si la inscripción se hubiera concretado. En cambio, con una checked exception, el compilador exige la presencia del bloque `try-catch`, asegurando que la UI despliegue una alerta semafórica adecuada al operador y evitando inconsistencias físicas en la capacidad de las instalaciones."*

BLOQUE 4: ALFONSINA — MEMBRESÍAS, INTERFAZ FACTURABLE Y BENEFICIOS

Tiempo asignado: 03 minutos 00 segundos (09:15 a 12:15)
Diapositivas: `Slide 07: Polimorfismo de Facturación, Antigüedad (10%) y Comprobantes`
Clases Java Clave: `com.clubdeportivo.model.Facturable`, `com.clubdeportivo.model.Membresia`, `com.clubdeportivo.model.Pago`, `com.clubdeportivo.service.FacturacionService`, `com.clubdeportivo.model.TipoMembresia`

Acotaciones Escénicas

- [09:15] *Alfonsina toma la palabra con dicción clara y ritmo pausado. Señala el contrato* `Facturable` *en la parte superior de la diapositiva.*
- [10:15] *Destaca con gestos la importancia de la regla de negocio de 12 meses de antigüedad y el cálculo exacto de la cuota.*
- [11:15] *Explica cómo la interfaz* `Facturable` *abre el sistema a futuras expansiones comerciales sin modificar código existente.*

Discurso de Exposición

[09:15 - 10:05] El Contrato Desacoplado: La Interfaz `Facturable`
"Muchas gracias Merlina. La sostenibilidad de cualquier club deportivo reside en su gestión financiera. Desde el inicio, nuestro equipo decidió que el módulo de cobranzas no debía estar acoplado exclusivamente a la figura del `Socio`. Para lograrlo, recurrimos a una de las herramientas más poderosas de la POO: la abstracción mediante interfaces.

Definimos la interfaz formal `Facturable`, la cual estipula cuatro métodos esenciales: 1. `calcularMontoTotal()`: Determina la tarifa neta final a facturar. 2. `generarComprobante()`: Emite una cadena estructurada que formaliza el recibo impositivo. 3. `estaAlDia()`: Retorna un booleano sobre la regularidad contable. 4. `aplicarDescuento(double porcentaje)`: Permite imputar bonificaciones comerciales.

Al hacer que `Socio` implemente `Facturable`, nuestro servicio `FacturacionService` puede liquidar cuotas de manera puramente polimórfica sin conocer detalles íntimos de la persona."

[10:05 - 11:00] Membresías y la Regla del 10% por Antigüedad
"Cada socio posee asociada una `Membresia`, tipificada mediante el enum `TipoMembresia` (`PASE_LIBRE`, `MUSCULACION`, `CLASES_GRUPALES`, `VIP_TOTAL`), cada una con su precio base estandarizado.

Sobre esta base impositiva, implementamos una regla de negocio fundamental exigida por la cátedra: **fidelización por antigüedad**. Al invocar `socio.calcularMontoTotal()`, el método realiza una introspección cronológica:

```
long mesesAntiguedad = ChronoUnit.MONTHS.between(fechaRegistro, LocalDate.now());
if (mesesAntiguedad ⩾ 12) {
    montoTotal = montoBase * 0.90; // 10% de descuento automático
}
```

Si un socio acumula más de 12 meses ininterrumpidos en el club, el sistema aplica automáticamente un descuento del 10%. Por ejemplo, una membresía VIP de $15.000 pasa automáticamente a costar $13.500.

Esta lógica fue validada con precisión en nuestro Test Unitario N° 4, contrastando tanto socios antiguos como nuevos."

[11:00 - 11:45] Emisión de Comprobantes y Transaccionalidad de Pagos
"Cuando se efectiviza un cobro, se instancia la entidad `Pago`, registrando número correlativo de recibo, marca temporal `LocalDateTime.now()`, método de pago (`EFECTIVO`, `TARJETA_DEBITO`, `TRANSFERENCIA`) y referencia al socio.

El método `generarComprobante()` emite una constancia digital auditada con código alfanumérico correlativo, actualizando en memoria el estado del socio a `EstadoCuota.AL_DIA`. Si un socio registra cuotas impagas, el sistema lo marca automáticamente como `PENDIENTE` o `VENCIDO`, inhabilitando su acceso físico en el molinete."

[11:45 - 12:15] Cierre y Pase a la Interfaz Visual
"Gracias a este diseño, el sistema está abierto a la extensión y cerrado a la modificación (Principio Open/Closed de SOLID): mañana el club podría vender suplementos o alquilar quinchos implementando `Facturable` sin alterar una sola línea del motor de facturación. Ahora, para descubrir cómo interactúa el personal del club con estas funciones, le doy la palabra a Sara con la interfaz gráfica Swing."

Defensa de Cátedra (Pregunta Típica del Jurado)

> **Profesor:** *"¿Por qué declararon `Facturable` como una `interface` en lugar de una clase abstracta? ¿Por qué no heredar de una clase `EntidadComercial`?"*
> **Respuesta de Alfonsina:** *"La decisión obedece a dos pilares fundamentales: la limitación de Java respecto a la herencia simple y el Principio de Segregación de Interfaces (ISP). La clase `Socio` ya hereda de `Persona`; si `Facturable` fuera una clase abstracta, Java nos impediría heredarla simultáneamente. Por otro lado, una interfaz representa una capacidad o contrato de comportamiento ('puede ser facturado'), no una relación taxonómica de identidad ('es un'). Al utilizar una interfaz, desacoplamos totalmente la facturación del árbol de personas: mañana podemos facturar el alquiler de una cancha de pádel o la venta de un pase diario de visita sin forzarlos a ser personas."*

BLOQUE 5: SARA — INTERFAZ GRÁFICA SWING, THEME CONFIG Y MOLINETE VIRTUAL

Tiempo asignado: 03 minutos 00 segundos (12:15 a 15:15)
Diapositivas: `Slide 08: Interfaz Gráfica Swing: CardLayout, ThemeConfig y Badges` | `Slide 09: Molinete Virtual y Control de Acceso`
Clases Java Clave: `com.clubdeportivo.gui.MainFrame`, `com.clubdeportivo.gui.theme.ThemeConfig`, `com.clubdeportivo.gui.components.StatusBadgeCellRenderer`, `com.clubdeportivo.gui.CheckInDialog`, `com.clubdeportivo.service.AccesoService`

Acotaciones Escénicas

- [12:15] *Sara avanza con entusiasmo y profesionalismo. Muestra en pantalla las capturas de la GUI real de FitManage Pro.*
- [13:10] *Destaca la transición sin parpadeos lograda gracias al CardLayout.*
- [14:05] *Avanza a la Slide 09 y describe con pasión el funcionamiento interactivo del simulador de molinete virtual.*

Discurso de Exposición

[12:15 - 13:00] Arquitectura GUI: Single Window y CardLayout
"Muchas gracias Alfonsina. Un software robusto pierde su valor si su interfaz de usuario resulta tosca, confusa o inestable. Para FitManage Pro diseñamos una experiencia visual de escritorio sofisticada, construida en **Java Swing nativo**, evitando la típica proliferación desordenada de múltiples ventanas `JFrame` flotantes.

Nuestra arquitectura visual descansa en la clase `MainFrame`, la cual implementa un panel lateral de navegación o *Sidebar* vinculado a un panel contenedor central gobernado por un **`CardLayout`**. Cada módulo funcional —Socios, Clases, Rutinas, Pagos y Asistencias— reside en su propio panel especializado (`JPanel`). Cuando el operador selecciona una opción en el menú, el `CardLayout` conmuta instantáneamente el panel visible en memoria, brindando una experiencia fluida, reactiva y limpia, exactamente idéntica a la navegación de una aplicación web moderna tipo SPA."

[13:00 - 13:50] Estandarización Visual: ThemeConfig y Badges Cromáticos
"Para erradicar la estética anticuada que suele asociarse a Swing, creamos la clase utilitaria `ThemeConfig`. Centralizamos en ella una paleta cromática profesional de alto contraste denominada *Obsidian Slate / Electric Blue*, fuentes tipográficas escaladas (`Segoe UI` / `SansSerif`) y constantes dimensionales para bordes y rellenos.

Asimismo, enriquecimos las tablas de datos (`JTable`) mediante componentes customizados. Desarrollamos el `StatusBadgeCellRenderer`, un renderizador de celdas que dibuja insignias redondeadas tipo píldora (*badges*) con semáforo cromático: - **Verde Esmeralda** para socios con cuota `AL_DIA`. - **Rojo Coral** para socios con cuota `VENCIDO`. - **Ámbar Dorado** para estados `PENDIENTE`. Esto permite que el recepcionista del club identifique la condición administrativa del socio en menos de 50 milisegundos de vistazo visual."

[13:50 - 14:45] El Molinete Virtual: `CheckInDialog` y Validación en Tiempo Real
"La culminación de nuestra interfaz es el **Simulador de Molinete Virtual (`CheckInDialog`)**. En un gimnasio real, el control de acceso en molinetes físicos es el punto de mayor fricción operativa.

Cuando se ingresa un DNI o número de socio en el diálogo, este delega la evaluación a `AccesoService.validarIngresoGeneral(dni)`. El servicio recupera al socio y consulta polimórficamente su estado: 1. Si el socio está `AL_DIA`, el molinete enciende un halo **Verde**, emite el sonido virtual de apertura, desbloquea el acceso e inserta una nueva `Asistencia` válida con marca temporal auditada. 2. Si el socio tiene cuota `VENCIDO`, el sistema pinta el semáforo en **Rojo**, muestra la deuda en pantalla y deniega el paso con estado `DENEGADO_CUOTA_VENCIDA`. 3. Si el socio intenta ingresar a una clase grupal particular, el sistema valida adicionalmente que posea membresía habilitante y que la sala no supere su aforo."

[14:45 - 15:15] Cierre y Pase a Persistencia y Calidad
"Toda esta interacción gráfica opera de manera desacoplada: los paneles jamás tocan archivos ni manipulan JSON; interactúan exclusivamente con la capa de Servicios. Para revelar el motor de datos subyacente que hace posible esta magia sin bases de datos externas, le cedo la palabra a Joel."

Defensa de Cátedra (Pregunta Típica del Jurado)

> **Profesor:** *"¿Por qué eligieron Swing con `CardLayout` en lugar de abrir ventanas `JDialog` o `JFrame` independientes? ¿Cómo evitan que la interfaz gráfica se freeze o congele al procesar datos?"*
> **Respuesta de Sara:** *"Elegimos `CardLayout` dentro de un único `MainFrame` para garantizar una experiencia de usuario cohesionada y profesional, evitando la saturación del escritorio con múltiples ventanas que compiten por el foco del usuario. Para evitar que la interfaz se congele, aplicamos el estricto respeto del EDT (Event Dispatch Thread*): las operaciones de interfaz se canalizan mediante `SwingUtilities.invokeLater()`, y los servicios de negocio resuelven sus consultas en memoria de forma inmediata gracias a los repositorios indexados por mapa hash. De haber operaciones de red pesadas, SwingWorker sería la solución natural, pero nuestro motor de persistencia atómica responde en menos de 5 milisegundos."*

BLOQUE 6: JOEL — PERSISTENCIA ATÓMICA JSON, TESTS, GRAPHIFY Y CIERRE

Tiempo asignado: 03 minutos 15 segundos (15:15 a 18:30)
Diapositivas: `Slide 10: Persistencia Atómica JSON` | `Slide 11: Tests Automatizados y Graphify` | `Slide 12: Trazabilidad y Defensa` | `Slide 13: Conclusiones y Cierre`
Clases Java Clave: `com.clubdeportivo.util.JsonPersistenceEngine`, `com.clubdeportivo.repository.IRepository`, `com.clubdeportivo.test.SistemaTest`, `scripts/test.sh`

Acotaciones Escénicas

- [15:15] *Joel se incorpora con aplomo técnico. Tono analítico y riguroso. Muestra el flujo de escritura segura en la Slide 10.*
- [16:05] *Pasa a la Slide 11, señalando el badge verde 7/7 tests y las métricas de Graphify.*
- [17:20] *Avanza a la Slide 12 y 13. El equipo completo se coloca a su lado para el saludo final.*

Discurso de Exposición

[15:15 - 16:15] Persistencia Atómica en JSON: `JsonPersistenceEngine`
"Muchas gracias Sara. Llegamos al corazón de la persistencia de FitManage Pro. La consigna de la cátedra prohibía el uso de bases de datos externas o librerías de terceros como Jackson o Gson. Debíamos desarrollar un motor autónomo utilizando **Java SE puro**.

Para ello creé `JsonPersistenceEngine`, un parser y serializador recursivo capaz de transformar grafos de objetos en archivos JSON legibles y viceversa. Pero enfrentábamos un riesgo crítico: ¿qué ocurre si el club sufre un corte de suministro eléctrico o una caída del sistema exactamente en el instante en que se está guardando `socios.json`? En un guardado tradicional, el archivo quedaría truncado y la base de datos completa se corrompería.

Para resolverlo, implementé el patrón de **Escritura Atómica en Dos Fases**: 1. El motor escribe la información en un archivo temporal con sufijo `.tmp` (`socios.json.tmp`). 2. Fuerza el vaciado de los búferes del sistema operativo al disco físico invocando `channel.force(true)`. 3. Una vez garantizada la integridad física del temporal, se ejecuta la operación:

```
Files.move(tempFile, targetFile, StandardCopyOption.ATOMIC_MOVE, StandardCopyOption.REPLACE_EXISTI
```

En sistemas de archivos modernos POSIX y NTFS, el renombramiento a nivel de inodo es atómico e indivisible: tarda microsegundos. El archivo previo jamás se destruye hasta que el nuevo está 100% asegurado."

[16:15 - 17:15] Suite de Pruebas Automatizadas (7/7 Tests) y Graphify
"La calidad de un sistema no se presume; se demuestra empíricamente. Diseñamos una suite de pruebas automatizadas independiente en `SistemaTest.java`, ejecutable mediante `scripts/test.sh`.

La suite valida sin frameworks externos los 7 requerimientos troncales: 1. Herencia y Polimorfismo en `Persona` y `Socio`. 2. Composición Fuerte en `Rutina` y cálculo de 4.720 kg de tonelaje. 3. Rechazo estricto de aforo y disparo de `CupoAgotadoException`. 4. Regla del 10% de descuento por antigüedad en `Facturable`. 5. Validación de semáforo en molinete virtual (`AccesoService`). 6. Persistencia JSON Round-Trip con

verificación atómica. 7. Autenticación y liquidación de honorarios docentes. Los 7 tests arrojan **100% de éxito [PASS] con cero fallos**.

Complementariamente, auditamos el código fuente con la herramienta de grafos de conocimiento **Graphify**. El grafo resultante procesó 852 nodos y 2.168 relaciones. El análisis de ciclos de dependencias arrojó exactamente cero ciclos entre paquetes (`gui → service → repository → model`), comprobando que se preservó una arquitectura en capas inmaculada."

[17:15 - 18:30] Conclusiones Finales y Cierre del Equipo
"Para concluir nuestra exposición: FitManage Pro demuestra que no se requieren frameworks pesados ni dependencias externas para construir un software de gestión deportiva de nivel profesional, mantenible, seguro y escalable.

Hemos aplicado con rigor supremo la abstracción ontológica, el encapsulamiento protegido, el polimorfismo dinámico forzado, la distinción formal entre agregación débil y composición fuerte, el manejo tipado de excepciones y la persistencia atómica.

A nombre de Juan, Rocio, Merlina, Alfonsina, Sara y quien les habla, les agradecemos profundamente su atención durante estos 18 minutos y quedamos a total disposición del tribunal evaluador para responder sus preguntas y proceder con la demostración interactiva en vivo. Muchas gracias."

Defensa de Cátedra (Pregunta Típica del Jurado)

> **Profesor:** *"¿Qué garantiza la persistencia atómica en su motor JSON? ¿Qué pasaría si la computadora se apaga exactamente mientras se escribe un archivo?"*
> **Respuesta de Joel:** *"El método `saveToFileAtomic()` escribe primero todo el contenido en un archivo temporal con sufijo `.tmp` invocando `channel.force(true)` para volcar el búfer de memoria a disco físico. Únicamente si esa escritura concluye con éxito, se ejecuta `Files.move(..., ATOMIC_MOVE)`. En sistemas de archivos compatibles con POSIX como Linux ext4, el renombre atómico es una operación del inodo que tarda microsegundos y es indivisible: o queda el archivo previo intacto o queda el nuevo. Es imposible que el archivo de datos (`socios.json` , `pagos.json`) quede truncado a la mitad o corrupto."*

BLOQUE 7: RONDA DE PREGUNTAS DEL JURADO Y BUFFER DE CONTINGENCIA

Tiempo remanente: 01 minuto 30 segundos a 02 minutos (18:30 a 20:00)
Diapositiva: `Slide 13: Conclusiones, Logros Técnicos y Ronda de Preguntas`

Matriz Rápida de Respuestas de Apoyo

ÁREA DE PREGUNTA	INTEGRANTE RESPONSABLE	CONCEPTO CLAVE A DEFENDER
¿Por qué Java SE puro sin Spring/Lombok?	Juan / Joel	Demostración didáctica de la mecánica interna de POO, JVM pura y ausencia de magia de frameworks.
¿Cómo agregan un nuevo tipo de ejercicio?	Rocio	Extensibilidad en `GrupoMuscular` (enum) y parametrización de carga en `Ejercicio`.
¿Qué pasa con inscripciones concurrentes?	Merlina	Métodos `synchronized` en `ClaseGrupal` para prevenir condiciones de carrera (*race conditions*).
¿Cómo se expande la facturación a otros productos?	Alfonsina	Crear clases como `ArticuloTienda` o `AlquilerCancha` que implementen `Facturable`.
¿Por qué no usar JavaFX en lugar de Swing?	Sara	Swing está 100% incorporado en el OpenJDK sin módulos externos `openjfx`, asegurando portabilidad inmediata.
¿Cómo escala la persistencia con 100.000 socios?	Joel	Migrar la implementación de `IRepository<T>` a JDBC/SQL sin alterar la capa de servicios ni la GUI.

Fin del Guion Oficial de Exposición.